Classification:

Logistics Classification	[12]:
	``python
	from sklearn.linear_model import LogisticRegression
	``
	[13]:
	``python
	from sklearn.linear_model import LogisticRegression
	from sklearn.metrics import f1_score, accuracy_score, roc_auc_score
	
	# Create the classifier
	clf = LogisticRegression(max_iter=1000, solver='lbfgs', multi_class='auto')
	
	# Fit the model
	clf.fit(X_train, y_train)
	
	# Make predictions
	y_pred = clf.predict(X_test)
	y_proba = clf.predict_proba(X_test)[:, 1] # probability for positive class
	
	# Evaluate
	print("Accuracy:", accuracy_score(y_test, y_pred))
	print("F1 Score:", f1_score(y_test, y_pred, average='weighted'))
	print("ROC AUC:", roc_auc_score(y_test, y_proba))
	``
	``
	Accuracy: 0.9924812030075187
	F1 Score: 0.9924946382275899
	ROC AUC: 1.0
	``
	Jupyter Lab Python 3 (ipykernel)

KNN	```python
	X_train = sc.fit_transform(X_train)
	X_test = sc.transform(X_test)
	``
	[12]:
	``python
	from sklearn.neighbors import KNeighborsClassifier
	``
	[13]:
	``python
	from sklearn.neighbors import KNeighborsClassifier
	from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
	
	clf = KNeighborsClassifier(n_neighbors=5)
	clf.fit(X_train, y_train)
	
	y_pred = clf.predict(X_test)
	
	# k-NN does not support predict_proba for some metrics, but sklearn KNN does
	y_proba = clf.predict_proba(X_test)[:, 1] # for binary classification
	
	print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
	print("\nClassification Report:\n", classification_report(y_test, y_pred))
	print("ROC AUC:", roc_auc_score(y_test, y_proba))
	``
	``
	Confusion Matrix:
	[[51 0]
	[8 74]]
	
	Classification Report:
	precision recall f1-score support
	
	False 0.86 1.00 0.93 51
	True 1.00 0.90 0.95 82
	
	accuracy 0.94 133
	macro avg 0.93 0.95 0.94 133
	weighted avg 0.95 0.94 0.94 133
	
	ROC AUC: 0.9992826398852224
	``
	[ ]:

SVM	[12]:
	``python
	from sklearn.svm import SVC
	``
	[14]:
	``python
	from sklearn.svm import SVC
	from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
	
	clf = SVC(kernel='rbf', probability=True) # probability=True needed for ROC AUC
	clf.fit(X_train, y_train)
	
	y_pred = clf.predict(X_test)
	y_proba = clf.predict_proba(X_test)[:, 1]
	
	print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
	print("\nClassification Report:\n", classification_report(y_test, y_pred))
	print("ROC AUC:", roc_auc_score(y_test, y_proba))
	``
	``
	Confusion Matrix:
	[[51 0]
	[1 81]]
	
	Classification Report:
	precision recall f1-score support
	
	False 0.98 1.00 0.99 51
	True 1.00 0.99 0.99 82
	
	accuracy 0.99 133
	macro avg 0.99 0.99 0.99 133
	weighted avg 0.99 0.99 0.99 133
	
	ROC AUC: 1.0
	``
	[ ]:

Decision Tree	[14]:
	``python
	from sklearn.tree import DecisionTreeClassifier
	from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
	
	clf = DecisionTreeClassifier(max_depth=None, random_state=42)
	clf.fit(X_train, y_train)
	
	y_pred = clf.predict(X_test)
	y_proba = clf.predict_proba(X_test)[:, 1]
	
	print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
	print("\nClassification Report:\n", classification_report(y_test, y_pred))
	print("ROC AUC:", roc_auc_score(y_test, y_proba))
	``
	``
	Confusion Matrix:
	[[50 1]
	[10 72]]
	
	Classification Report:
	precision recall f1-score support
	
	False 0.83 0.98 0.90 51
	True 0.99 0.88 0.93 82
	
	accuracy 0.92 133
	macro avg 0.91 0.93 0.91 133
	weighted avg 0.93 0.92 0.92 133
	
	ROC AUC: 0.9292204868675275
	``
	[ ]:

Naive Bayes	[15]:
	``python
	from sklearn.naive_bayes import GaussianNB
	# Import metrics
	from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
	
	clf = GaussianNB()
	clf.fit(X_train, y_train)
	
	y_pred = clf.predict(X_test)
	y_proba = clf.predict_proba(X_test)[:, 1]
	
	print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
	print("\nClassification Report:\n", classification_report(y_test, y_pred))
	print("ROC AUC:", roc_auc_score(y_test, y_proba))
	``
	``
	Confusion Matrix:
	[[51 0]
	[3 79]]
	
	Classification Report:
	precision recall f1-score support
	
	False 0.94 1.00 0.97 51
	True 1.00 0.96 0.98 82
	
	accuracy 0.98 133
	macro avg 0.97 0.98 0.98 133
	weighted avg 0.98 0.98 0.98 133
	
	ROC AUC: 1.0
	``
	[ ]:

Gradient Boosting	[12]: from sklearn.ensemble import GradientBoostingClassifier [13]: from sklearn.ensemble import GradientBoostingClassifier # Import metrics from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score clf = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42) clf.fit(X_train, y_train) y_pred = clf.predict(X_test) y_proba = clf.predict_proba(X_test)[:, 1] print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred)) print("\nClassification Report:\n", classification_report(y_test, y_pred)) print("ROC AUC:", roc_auc_score(y_test, y_proba)) Confusion Matrix: [[51 0] [1 81]] Classification Report: precision recall f1-score support False 0.98 1.00 0.99 51 True 1.00 0.99 0.99 82 accuracy 0.99 133 macro avg 0.99 0.99 0.99 133 weighted avg 0.99 0.99 0.99 133 ROC AUC: 1.0
Quadratic Discriminant Analysis (QDA)	[13]: from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis # Import metrics from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score clf = QuadraticDiscriminantAnalysis() clf.fit(X_train, y_train) y_pred = clf.predict(X_test) y_proba = clf.predict_proba(X_test)[:, 1] print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred)) print("\nClassification Report:\n", classification_report(y_test, y_pred)) print("ROC AUC:", roc_auc_score(y_test, y_proba)) Confusion Matrix: [[51 0] [1 81]] Classification Report: precision recall f1-score support False 0.98 1.00 0.99 51 True 1.00 0.99 0.99 82 accuracy 0.99 133 macro avg 0.99 0.99 0.99 133 weighted avg 0.99 0.99 0.99 133 ROC AUC: 0.9992826398852224 C:\Users\ASUS\anaconda3\Lib\site-packages\sklearn\discriminant_analysis.py:926: UserWarning: Variables are collinear warnings.warn("Variables are collinear")

Summary :
For the CKD dataset using classifier Algorithms the below list of algorithms are giving more accurate ROC value as 1
=>SVM
=>Gradient Boosting
=>Navie Bayes
=>Logistics

Using Grid:

Name	Clf Report
Logistics Grid Classification	[21]: print("The report:\n",clf_report) The report: precision recall f1-score support False 0.98 1.00 0.99 51 True 1.00 0.99 0.99 82 accuracy 0.99 133 macro avg 0.99 0.99 0.99 133 weighted avg 0.99 0.99 0.99 133
RF Grid Classification	[16]: from sklearn.metrics import classification_report clf_report = classification_report(y_test, y_pred) [17]: print(clf_report) precision recall f1-score support False 0.98 0.98 0.98 51 True 0.99 0.99 0.99 82 accuracy 0.98 133 macro avg 0.98 0.98 0.98 133 weighted avg 0.98 0.98 0.98 133

DF Classification	[21]: `print("The report:\n",clf_report)` The report: precision recall f1-score support False 0.98 1.00 0.99 51 True 1.00 0.99 0.99 82 accuracy 0.99 133 macro avg 0.99 0.99 0.99 133 weighted avg 0.99 0.99 0.99 133 [20]: `from sklearn.metrics import roc_auc_score` `roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])` [20]: 0.9939024390243902

SVM Grid	[18]: `print("The report:\n",clf_report)`
	The report:
	precision recall f1-score support
	False 0.98 1.00 0.99 51
	True 1.00 0.99 0.99 82
	accuracy 0.99 133
	macro avg 0.99 0.99 0.99 133
	weighted avg 0.99 0.99 0.99 133
	[20]: `from sklearn.svm import SVC`
	`from sklearn.model_selection import GridSearchCV`
	`from sklearn.metrics import roc_auc_score`
	`# Grid search with probability enabled`
	`param_grid = {`
	`'C': [0.1, 1, 10],`
	`'kernel': ['linear', 'rbf'],`
	`'gamma': ['scale', 'auto']`
	`}`
	`grid = GridSearchCV(SVC(probability=True), param_grid, refit=True, verbose=3, n_jobs=-1, scoring='f1_weighted')`
	`# Fit model`
	`grid.fit(X_train, y_train)`
	`# Predict probabilities`
	`y_proba = grid.predict_proba(X_test)[:, 1]  # probability for positive class`
	`# ROC AUC`
	`roc_score = roc_auc_score(y_test, y_proba)`
	`print("ROC AUC:", roc_score)`
	Fitting 5 folds for each of 12 candidates, totalling 60 fits
	ROC AUC: 0.9992826398852224

Summary:
For the CKD dataset all the above algorithms are working accurately .